University of Gujrat

Assginment

Submitted By: Abdullah Khan

Roll No: [25011598-062]

Submitted To: Mr Muhammad Sabeeh.

Course Title: Data Structures

Course Code: DS - 202

Section: Data Science (A)

Submission Date:

CamScanner

Frequency Count Method

Algorithm of Sum:

```
Algorithm Sum (A, n)
{
    S = 0;                          1
    for (i=0; i<n; i++)             n+1
    {
        S = S + A[i];               n
    }
    return S;                       1
}
```

Time: $f(n) = 2n+3$

$O(n)$

Space complexity

A → n

n → 1

S → 1

i → 1

$S(n) = n+3$

$O(n)$

Algorithm of Sum of matrices

```
Algorithm Add (A, B, n)
{
    for (i=0; i<n; i++)             n+1
```

```
{
    for (j=0; j<n ; j++)          n x (n+1)
    {
      C[i,j] = A[i,j] + B[i,j];  -> n x n
    }
}
```

Time = $f(n) = 2n^2 + 2n + 1$

$O(n^2)$

Space:

A — n^2

B — n^2

C — n^2

n — 1

i — 1

j — 1

$S(n) = 3n^2 + 3$

$O(n^2)$

Algorithm of Multiply

```
Algorithm Multiply (A, B, n)
{
    for (i=0; i<n ; i++)          n+1
    {
      for (j=0; j<n; j++)         (n+1)n
      {
        C[i,j] = 0;               n x n
      for (k=0; k<n ; k++)        (n+1) x n x n
```

```
    {
C[i,j]=C[i,j] + A[i,k] * B[k,j]; n*n*n
    }
  }
 }
}
```

Time = $f(n) = 2n^3 + 3n^2 + 2n + 1$

$O(n^3)$

Space:

$A = n^2$

$B = n^2$

$C = n^2$

$n = 1$

$j = 1$

$i = 1$

$k = 1$

$S(n) = 3n^2 + 4$

$O(n^2)$

Time — $O(n^3)$

Space — $O(n^2)$

Time complexity

```
for (i=0; i<n ; i++)        n+1
  {
    stmt;                   n
  }
```

$O(n)$

---e(v)e---

```
for (i=0; i<n; i++)  ~~~~ n+1
  {
    for (j=0; j<n; j++)  ⟶ n x (n+1)
      {
        stmt;  ⟶ n x n
      }
  }
```

Space: Time: $f(n) = 2n^2 + 2n + 1$

Space $O(n^2)$

---e(vi)e---

```
for (i=0; i<n; i++)
  {
    for (j=0; j<i; j++)
      {
        stmt;
      }
  }
```

i	j	time
0	0	0
1	0	1
2	0	2
3	3	3
|		|
n		n

Time: $f(n) = \frac{n^2 + 1}{2}$

Space: $O(n^2)$

--(Vii)--

Complexity

```
p = 0 ;
for (i = 1 ; p <= n ; i++)
{
    p = p + i ;
}
```

i	p
1	0+1 = 1
2	1+2 = 3
3	1+2+3
4	1+2+3+4
⋮	⋮
K	1+2+3+4+...+K

Assume $p > n$

$$p = \frac{K(K+1)}{2}$$

Time: $\frac{K(K+1)}{2} > n$

$K^2 > n$

$K > \sqrt{n}$

Space $O(\sqrt{n})$

--(Viii)--

(A) Complexity

```
for (i = 1 ; i < n ; i = i * 2)
{
    stmt ;
}
```

i
1
1 x 2 = 2
2 x 2 = 4
$2^2 \times 2 = 8$
⋮
2^K

Assume $i >= n$

$\therefore i = 2^K$

Time: $2^K >= n$

$2^K = n$

$K = \log_2 n$ $\quad O(\log_2 n)$

```
for (i=1; i<=n; i++)
{
  stmt;
}
```

$i = 1+1+1+1+\dots+1 = n$

$K = n$

(B)

```
for (i=1; i<n; i=i*2)
{
  Stmt;          — log n
}
```

Time $[\log n]$

$n = 8$		$n = 10$	
i		i	
1	3	1	4
2		2	
4		4	
8x		8	
		16	

$\log 8 = 3$

$\log_2 2^{3*}$

$3 \log_2 2$

$\log_2 10$

$3 \cdot 2$

Time 3

Complexity

```
for (i = n ; i >= 1 ; i = i/2)
{
    Stmt ;
}
```

Assume $i < 1$

$\therefore \frac{n}{2^k} < 1$

$\frac{n}{2^k} = 1$

$n = 2^k$

$k = \log_2 n$

Time:

Space: $O(\log_2 n)$

i
n
$\frac{n}{2}$
$\frac{n}{2^2}$
$\frac{n}{2^3}$
⋮
$\frac{n}{2^k}$

Loop

```
for (i=0 ; i*i<=n ; i++)
{
    stmt ;
}
```

$i * i < n$

$i * i \geq n$

$i^2 = n$

$i = \sqrt{n}$

```
for (i=0; i<n; i++)
{
    stmt;                    n
}
for (j=0; j<n; j++)
{
    stmt: ——————             n
}
```

Time: $f(n) = 2n$

Space: $O(n)$

```
p = 0
for (i=1; i<n; i=i*2)
{
    p++;                     log n
}
for (j=1; j<p; j=j*2)
{
    stmt;                    log p
}
```

Space $O(\log \log n)$

...Q(iii)...

```
for (i=0; i<n; i++)                    n
{
    for (j=1; j<n; j=j*2)              n x log n
    {
        stmt;                          n x log n
    }
}
```

Time = $f(n) = 2n \log n + n$

Space = $O(n \log n)$

...Q(iv)...

```
i = 0;                                 1
while (i<n)                            n+1
{
    Stmt;                              n
    i++;                               n
}
```

Time = $3n + 2$.

Space = $O(n)$

...Q(v)...

```
a = 1                                  a
while a<b                              1
{                                      1 x 2 = 2          Terminate
    stmt;                              2 x 2 = 2^2        a >= b
    a = a*2;                           2^2 x 2 = 2^3      a = 2^k
}                                      ↓                  2^k >= b
                                       2^k                2^k = b
```

```
for (a=1; a<b; a=a*2)
{
    stmt;
}
```

$K = \log_2 b$

Space = $O(\log n)$

```
i = 1;
K = 1;
while (K<n)
{
    stmt;
    K = K+i
    i++;
}
```

```
for (K=1; i=1; K<n; i++)
{
    stmt;
    K = K+i;
}
```

Space = $O(\sqrt{n})$

```
Algorithm Test(n)
{
    if (n<5)
    {
        printf("%d", n);
    }
    else
    {
        for (i=0; i<n; i++)
        {
```

```
        printf ("%d", i);
    }
}
```

Bubble Sort

Bubble (DATA, N)

1. Repeat for K = 1 to N - 1
2. Set PTR = 1
3. Repeat while PTR <= N - K
4. If Data [PTR] > Data [PTR+1] then
5. Interchange Data PTR and Data [PTR+1]
6. End If
7. Set PTR = PTR + 1
8. End Repeat
9. End Repeat
10. Exit

let tak = [5, 1, 3, 4, 2] N = 5

Original: 5 1 3 4 2

Pass 1: 1 3 4 2 5

Pass 2: 1 3 2 4 5

Pass 3: 1 2 3 4 5

Pass 4: 1 2 3 4 5

Sorted 1 2 3 4 5

--Quiz--

Selection sort

Selection Sort (A, n)

1. Set U = 1
2. Repeat while U < n
3. set Temp = A[U]
4. set loc = U
5. set I = U+1
6. Repeat while I <= n
7. If temp > A[I] then
8. Temp = A[I]
9. Loc = I
10. End If
11. Set I = I+1
12. End repeat
13. set T = A[Loc]
14. A[loc] = A[U]
15. A[U] = T
16. Set U = U+1
17. End Repeat
18. Exit

A = [4, 19, 1, 3]

N = 4

since

N-1 = 4 - 1 = 3

Iteration 1 - V = 1

Array:

[4, 19, 1, 3]

Tem = 4

Loc = 1

4 < 19

4 > 1

so

Temp = 1

Loc = 3

1 < 3

Now swap

A[1] ↔ A[3]

Now

[1 19, 4, 3]

Iteration 2 - V = 2

[1, 19, 4, 3]

19, 4, 3

Temp = 19

Loc = 2

19 > 4

So

Temp = 19

loc = 3

4 > 3

So

Now swap

A[2] ↔ A[4]

Now

[1, 3, 4, 79]

Iteration 3 - U = 3

[1, 3, 4, 79]

{ 4, 79

Temp = 4

loc = 3

4 < 79

temp = 4

loc = 3

A[3] ↔ A[3]

Array

[1, 3, 4, 79]

Iteration	smallest	swap	Array
Original	—	—	4, 79, 1, 3
1	1	A[1] ↔ A[3]	1, 79, 4, 3
2	3	A[2] ↔ A[4]	1, 3, 4, 79
3	4	A[3] ↔ {A[3]	1, 3, 4, 79